MOVEINSYNC · FORWARD DEPLOYED ENGINEER ASSIGNMENT

# Where the 1-star reviews come from, and what to fix first

A public-data investigation of the employee app on Google Play, a ranked fix list, a first engineering ticket and a weekly process to keep asking the question.

Shreya S · Reg. no. 12320371 · 12 October 2026 · Public sources only, no internal data

# The rating is not low across the board. A block of 1-star reviews drags it down.

**FACT**

About 114K ratings average 4.31. Of those, 12.3% are 1-star and only 2.7% are 2 or 3-star. The distribution is polarised.

**FINDING (SMALL SAMPLE)**

Critical reviews cluster at gate moments: login OTP, boarding OTP behind a seat-belt step, "is my trip booked?", and a no-show that arrives as a surprise.

**RECOMMENDATION**

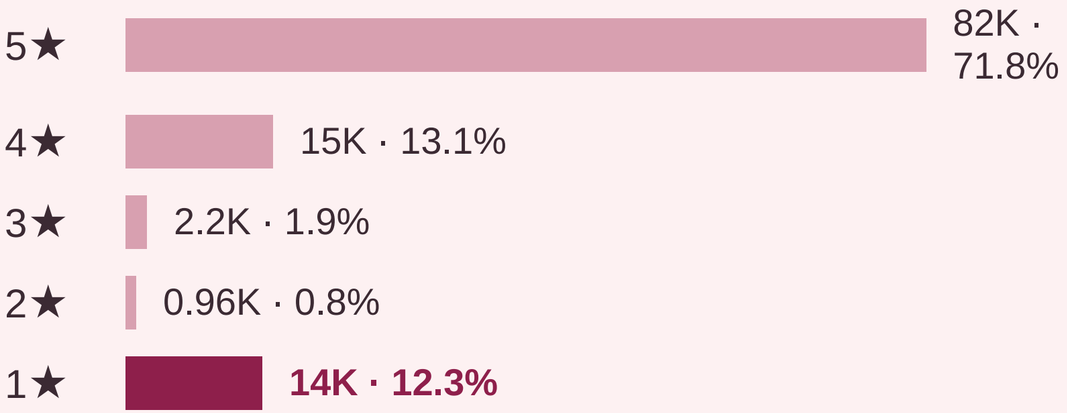
Fix the gates in order, starting with the boarding OTP because it touches safety. Then run a weekly loop from review to ticket to verified fix.

Caveat: I could read only 13 Android reviews (8 negative). The pattern is a lead to confirm with internal data in the first three days, not a conclusion.

Sources: Google Play listing, AppBrain histogram (third party) · 2

# One in eight ratings is 1-star, and peers look similar or worse

## MoveInSync employee app, ratings by star (K)




## Same lens on three comparable commute apps

| App | Avg | 1★ share | Ratings |
|---|---|---|---|
| MoveInSync | 4.32 | 12.3% | 114.6K |
| Safetrax | 4.21 | 12.1% | 24.1K |
| Routematic | 3.81 | 17.9% | 34.8K |
| eFmFm | 3.50 | 35.1% | 4.9K |

Open question: the Play listing shows 4.6 while the histogram implies 4.31. Confirm which figure leadership tracks.

Illustrative arithmetic: if half of today's 1-star ratings became 4-star, the average would move from about 4.31 to about 4.50. Gains come from a specific block, not general polish.

Source: appbrain.com pages for each app (histograms are rounded, shares computed by me) · 3

# Six patterns in 8 negative reviews, each tied to a journey stage

| Pattern | Evidence (paraphrased, 2026) | Stage | Count | Conf. |
|---|---|---|---|---|
| Boarding OTP gated and offline-fragile | 1★ 30 Sep: seat-belt check hides OTP, app needs network each open. Help Center confirms OTP shows after the seat-belt step. | Boarding | 1 of 8 | Low-Med |
| Booking state unclear | Play 7 Jul: shuttle booking became two steps, reviewer thought a cab was booked. 1★ 6 Oct: booked trip, push says not booked. | Booking | 2 of 8 | Medium |
| No-show arrives as a surprise | Play 29 Jun, 6 helpful votes: missed check-in after a late cab is marked no-show. | Pickup | 1 of 8 | Low-Med |
| Login OTP not received | 1★ 24 Sep: repeated tries, no OTP. Help Center and a public forum post describe the same. | Onboarding | 1 of 8 | Medium |
| Tracking and ETA inaccurate | Play 1 Jul: slow GPS, wrong pickup and drop times. Peer app reviews report the same. | Live trip | 1 of 8 | Medium |
| Outside the app | 1★ 14 and 18 Sep: pickup location, routing and vendor service, not the UI. | Operations | 2 of 8 | Medium |

Also seen: a crash complaint (1★ 18 Sep, plus iOS reports after v10.42.x). Counts overlap, so they do not sum to 8. Sample: 3 Play listing reviews and 10 AppBrain comments, English only. No developer replies were visible.

Sources: play.google.com listing, appbrain.com comments, helpdesk help center · 4

# Trust breaks where the app is a gate between the employee and the cab

GATE
**Login**
OTP never arrives

GATE
**Booking**
Booked or not? Mixed signals

GATE
**Pickup**
Missed check-in becomes a no-show

FRICTION
**Live trip**
GPS and ETA drift

GATE · SAFETY
**Boarding**
OTP behind seat-belt step

NO VOICE
**Resolution**
No owner or update seen

### Hypothesis to test

A gate failure is silent and the employee has no other route or choice of app, so the Play review becomes their only complaint channel.

### What would prove it wrong

Internal data showing gate failures are rare per trip, or 1-star reviewers mostly citing routing and vendor service rather than app steps.

# Fix the boarding gate first, then booking, login and no-shows

Score = Frequency × Severity × Confidence ÷ Effort, each rated 1 to 5. Ratings are my provisional judgement on a small sample. Safety-linked items go first whatever the score.

| # | Action | F | S | C | E | Score | When |
|---|---|---|---|---|---|---|---|
| 1 | Boarding gate: OTP always reachable, trip cached offline, seat-belt prompt kept but never blocking | 3 | 5 | 3 | 3 | 15 | Now, safety first |
| 2 | Booking state: one clear confirmed status, remove contradictory "not booked" push | 3 | 4 | 3 | 2 | 18 | Now |
| 3 | Login OTP fallback: resend timer, alternate channel, helpdesk link | 2 | 5 | 3 | 2 | 15 | Now |
| 4 | No-show warning when the cab reaches the pickup point, with the check-in window shown | 2 | 4 | 3 | 2 | 12 | Medium term |
| 5 | Stale-ETA indicator ("updated 40s ago") instead of a confident wrong time | 3 | 3 | 3 | 3 | 9 | Medium term |
| 6 | Crash and ANR instrumentation; release notes that name what was fixed | 2 | 4 | 2 | 3 | 5 | Start week 1 |

Longer term: fix the GPS and ETA pipeline once internal telemetry shows where it drifts, and send routing and vendor complaints to transport desks with a visible owner.

# Ticket BOARD-1: the employee can always see the boarding OTP

**Problem.** A review says the seat-belt step hides the OTP and the app needs a network call every time it opens, which fails underground.

**Evidence.** Android 1★ dated 30 Sep 2026. Help Center: the OTP shows only after the seat-belt action, and the popup repeats at intervals.

**Reproduce (to confirm).** Test tenant with seatBeltNotificationEnabled on. Open an assigned trip, tap View OTP, then repeat in airplane mode and on a throttled network.

**Expected.** Last synced trip and OTP show with a "last updated" time. The seat-belt prompt takes one tap, and if it cannot load, the OTP still shows.

**Acceptance.** OTP visible within 2 taps. Offline open shows the cached trip. Seat-belt answer is still logged. Safety owner signs off the fallback wording.

**Instrumentation.** Events for otp_view_tapped, seatbelt_prompt_shown, seatbelt_prompt_answered, otp_displayed, time_to_otp_ms, offline_open, cache_hit.

**Tests.** 1) Prompt service times out, OTP still shows. 2) Airplane mode, cached trip opens. 3) Prompt answered, event logged once. 4) Cache older than the trip, stale banner shows. 5) Trip changed on the server, app refreshes on reconnect.

Written from public evidence. Names of flags and events are proposals to match to the real codebase.

# A 30-day plan that validates first, then ships the gate fixes

| When | What happens | Owner (role) |
|---|---|---|
| Days 1 to 3 | Request Play Console reviews and ratings, crash data, OTP and booking event logs. Confirm which rating figure counts. | FDE with product analyst |
| Days 4 to 7 | Check each hypothesis against internal data. Start crash and event instrumentation. Agree the seat-belt fallback with safety. | FDE, mobile lead, safety owner |
| Week 2 | Build BOARD-1, the booking status fix and the login fallback. Test on a staging tenant. | Mobile and backend engineers, QA |
| Week 3 | Staged rollout to a share of employees at willing customers. Compare with holdout. Reply to affected reviews. | Mobile lead, support lead |
| Week 4 | Read results, keep or roll back, write the next cycle from the first weekly report. | FDE, product manager |

Assumption: roles are generic because I do not know MoveInSync's team structure. Dates assume internal data access arrives in the first three days.

# Judge the fix by boarding outcomes first, stars last

### Public metrics (available now)

Average rating and weekly review volume. Share of 1-star reviews. Reviews tagged to each journey stage. Complaint recurrence by app version once versions are visible.

### Internal metrics (data requests)

Time to OTP displayed, OTP and booking failure rates, crash-free sessions, support contacts per trip, issue resolution time.

### First experiment: BOARD-1

**Primary:** p90 time to OTP displayed and share of boardings with an OTP failure.

**Guardrails:** seat-belt confirmation rate, crash-free sessions, boarding support contacts.

**Compare:** staged rollout against a holdout of employees in the same company, 2 weeks.

**Roll back** if seat-belt confirmation falls more than 2 points (provisional threshold).

Public reviews over-represent angry users and arrive late, and review volume can change for reasons unrelated to quality. Treat the star average as a lagging check, not the test.

# A weekly loop turns each complaint cluster into an owned, tested fix

**1 Collect**

Play Console export or API (data request), plus a dated listing snapshot

**2 Clean**

Deduplicate, detect language, keep the version if present

**3 Tag**

Rules for stage and failure, model for unclear cases, human check

**4 Fingerprint**

Stage, failure, version, recurrence, severity per cluster

**5 Act**

Draft QA test and ticket, assign an owner, report to leadership

**6 Verify**

Compare before and after, then feed the next week

## Rules or a model?

Rules handle stage keywords, versions, dedupe and language. A model helps with mixed or Hinglish reviews. Pin the prompt and model, store outputs, audit a sample by hand.

## Access limits

No scraping of undocumented endpoints. Until Play Console access exists, the weekly snapshot is manual and small, and the report says so.

# A first triage script already runs, and a customer team can use it this week

### Output on the 13-review sample

| Stage / failure | Reviews |
|---|---|
| Booking / state unclear | 2 |
| Operations / routing, vendor | 2 |
| Pickup / no-show surprise | 1 |
| Live trip / ETA inaccurate | 1 |
| Boarding / OTP gated | 1 |
| Onboarding / login OTP | 1 |
| App stability / crash | 1 |

### What it does

Reads a review export, tags stage and failure with rules, skips 4 and 5-star praise, and writes a tagged CSV, a one-page report and a draft QA test for each top failure.

### What it is not yet

Not validated yet. The rules were written while reading these same 13 reviews, so matching my manual tags proves nothing. A first run caught and fixed a praise review wrongly counted as a complaint.

### Next with the customer

Run it on the Play Console export and compare with an analyst's tags on 100 reviews before anyone trusts the counts.

Prototype: review_triage.py, rules only, no model calls · run on reviews_sample.csv · 11

# Appendix: how this was produced and what it cannot tell you

## Method and tools

Web search and page fetches of the official Play listing (com.moveinsync.ets), AppBrain pages, the MoveInSync Help Center and other public pages. A short Python calculation for the averages and shares. Reading and drafting were done with Claude. Tagging was done by hand, then a small rules-only Python script was written and run on the sample.

Sample: 3 reviews on the Play listing (29 Jun to 7 Jul 2026) and 10 AppBrain comments (14 Sep to 6 Oct 2026). 13 Android reviews, 8 negative, English only. Paraphrases are in reviews_sample.csv.

## Limits and assumptions

No full review history, no app version per review, and no developer replies were visible. The histogram is third party and rounded. Play shows 4.6 where the histogram implies 4.31.

Assumptions: employees often cannot choose another app, and a review mention is not the same as how often a failure happens. iOS reviews and peer apps were used only as supporting hints. The scores and experiment are proposals, not run. The script is unvalidated.